Facultad de ingeniería, Arquitectura y Diseño

Lic. En Sistemas Computacionales Con Énfasis En Desarrollo

De Sistemas Avanzados En Redes Y Software.

Base De Datos II

Prof. Maryon Torres

PROYECTO FINAL

Cristian Diaz PN-0048541

Roberto Lasso 8-1034-727

Carlos Poveda 8-976-1952

Josie Rodríguez 3-754-1856

Simulación de Sistema Producer–Consumer tipo Kafka

Introducción

El presente proyecto tiene como objetivo el desarrollo de un sistema integral de procesamiento de datos en tiempo real, aplicando los conceptos fundamentales de bases de datos relacionales, integración de APIs, procesamiento de eventos, ETL, visualización de datos y pruebas de rendimiento.

La solución propuesta implementa una arquitectura basada en eventos inspirada en Apache Kafka, permitiendo el manejo eficiente de transacciones de compra dentro de un sistema de comercio electrónico.

El sistema combina múltiples tecnologías para lograr una plataforma funcional, escalable y desacoplada, acercándose a entornos reales utilizados en la industria.

2. Caso de Uso

2.1 Descripción

Se desarrolló un sistema de tienda en línea que permite:

- Consultar productos disponibles
- Gestionar un carrito de compras
- Procesar pedidos
- Consultar historial de compras
- Visualizar estadísticas de ventas

2.2 Objetivo

Optimizar el procesamiento de compras mediante un enfoque asincrónico que permita:

- Mejorar el rendimiento del sistema
- Evitar bloqueos en procesos críticos
- Escalar el procesamiento de pedidos

2.3 Flujo del Sistema

1. El usuario realiza una compra desde el frontend
2. La API REST recibe la solicitud
3. Se genera un evento de compra
4. El evento es enviado a Kafka
5. Un consumidor procesa el evento
6. Se ejecuta la lógica de negocio en la base de datos
7. Se actualizan estadísticas y dashboard

3. Arquitectura del Sistema

3.1 Componentes

- API REST (Backend)
- Productor de eventos
- Consumidor de eventos
- Base de datos relacional
- Proceso ETL
- Interfaz Web (Frontend)
- Dashboard de visualización

3.2 Características

- Procesamiento en tiempo real
- Arquitectura desacoplada

- Escalabilidad horizontal
- Separación de responsabilidades

4. Base de Datos Relacional

4.1 Motor Utilizado

Se utilizó Microsoft SQL Server como sistema gestor de base de datos.

Justificación

Se seleccionó este motor debido a:

- Soporte para procedimientos almacenados complejos
- Uso de Table-Valued Parameters (TVP)
- Manejo eficiente de transacciones
- Integración con aplicaciones backend

4.2 Modelo de Datos

El diseño se realizó aplicando normalización hasta la Tercera Forma Normal (3FN):

- Eliminación de redundancias
- Dependencias funcionales correctas
- Separación de entidades

Tablas principales

- Cliente
- Producto
- Venta
- DetalleVenta

4.3 Vistas

Se implementaron vistas para facilitar consultas analíticas:

Vista 1: Productos más vendidos

```
CREATE VIEW vista_top_productos AS
SELECT p.Nombre, SUM(d.Cantidad) AS TotalVendido
FROM DetalleVenta d
JOIN Producto p ON p.IdProducto = d.IdProducto
GROUP BY p.Nombre;
```

Vista 2: Resumen de ventas

```
CREATE VIEW vista_resumen_ventas AS
SELECT COUNT(*) AS TotalVentas,
    SUM(Total) AS IngresosTotales
FROM Venta;
```

4.4 Procedimientos Almacenados

Se implementaron procedimientos para encapsular la lógica de negocio:

- **ProcesarCompra** - registra ventas y detalle
- **ObtenerDashboard** - estadísticas del sistema
- **ObtenerVentasCliente** - historial por cliente

5. API REST

5.1 Tecnología

Se utilizó Express.js para la implementación del backend.

5.2 Endpoints

Productos

- GET /productos → lista de productos

Dashboard

- GET /dashboard → estadísticas

Historial

- GET /historial/:email → compras del cliente

Compras

- POST /comprar → procesamiento asincrónico con Kafka
- POST /comprar-sync → procesamiento directo

5.3 Características

- Arquitectura RESTful
- Manejo de errores HTTP
- Integración con base de datos
- Uso de procedimientos almacenados

6. Integración con Kafka

Se implementó un sistema de mensajería utilizando Apache Kafka.

6.1 Producer

- Envía eventos de compra al tópico pedidos

6.2 Consumer

- Recibe eventos
- Procesa pedidos
- Ejecuta lógica de negocio
- Inserta datos en la base de datos

6.3 Beneficios

- Desacoplamiento del sistema
- Procesamiento asíncrono
- Mayor escalabilidad

7. Proceso ETL

7.1 Descripción

Se implementó un proceso ETL para la carga y actualización de productos.

7.2 Fases

Extract

- Lectura de datos desde archivo JSON

Transform

- Limpieza de datos
- Conversión de tipos
- Validación de registros

Load

- Inserción o actualización en la base de datos

7.3 Automatización
El proceso ETL se ejecuta mediante scripts en Node.js.

8. Visualización de Datos
8.1 Dashboard Web
El sistema incluye un dashboard interactivo que muestra:
- Total de ventas
- Ingresos generados
- Número de clientes
- Productos más vendidos

8.2 Herramientas BI
Se propone la integración con Power BI para:
- Análisis avanzado
- Reportes dinámicos
- Visualización en tiempo real

9. Pruebas del Sistema
9.1 Pruebas Funcionales
Se realizaron pruebas utilizando Postman para validar:
- Correcto funcionamiento de endpoints
- Manejo de datos
- Respuestas del sistema

9.2 Pruebas de Carga
Se utilizó Apache JMeter.

Escenarios
- Simulación de múltiples usuarios
- Comparación entre:
  - Procesamiento con Kafka
  - Procesamiento síncrono

Resultados
- Kafka reduce la carga directa en la API
- Mejora la concurrencia
- Permite mayor escalabilidad

10. Manual de Uso
Pasos para ejecutar el sistema
1. Iniciar el motor de base de datos
2. Ejecutar scripts de creación de tablas y procedimientos
3. Iniciar el servidor de Kafka
4. Ejecutar el consumer
5. Ejecutar la API REST
6. Abrir la interfaz web en el navegador

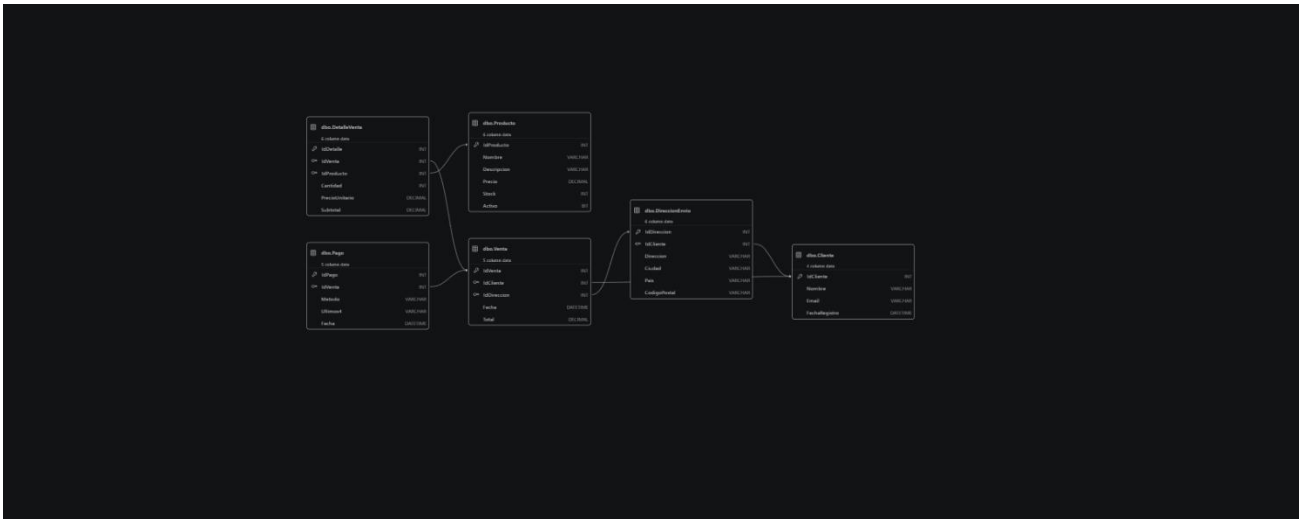
11. Lecciones Aprendidas
- Las arquitecturas basadas en eventos mejoran la escalabilidad

- Kafka permite desacoplar los componentes del sistema
- El uso de procedimientos almacenados optimiza la lógica de negocio
- ETL garantiza calidad y consistencia de datos
- Las pruebas de carga son esenciales para validar rendimiento

12. Glosario

- **Kafka**: Plataforma de transmisión de eventos
- **Producer**: Generador de eventos
- **Consumer**: Procesador de eventos
- **ETL**: Proceso de extracción, transformación y carga
- **API REST**: Interfaz basada en HTTP para comunicación entre sistemas

15. ERD



Mas Visible en el SVG que se encuentra en el github.

14. Conclusión

El proyecto desarrollado demuestra la integración efectiva de múltiples tecnologías para la construcción de un sistema de procesamiento de datos en tiempo real.

Se logró implementar una solución completa que incluye base de datos, API REST, procesamiento de eventos, ETL, visualización y pruebas, cumpliendo con los objetivos del curso y acercándose a soluciones utilizadas en entornos empresariales.